CityFM is one model with two encoders, trained in three stages

Leeds v5. The three stages train parts of one network, which is saved as one file (Leeds_v5/Model/model.pkl). Embeddings are computed afterwards, with the model frozen.

T text encoder · V footprint encoder · P position code (not in the model) · trained in that stage · frozen in that stage · ⟷ made more similar during training

1 The model

One network, saved in one file. It has two encoders, and each turns its input into a list of 768 numbers. Position is encoded separately, by a fixed formula outside the model.

model.pkl · the trained CityFM model

T · text encoder model.lm, model.projection_p

tags as text ▶ BERT-base ▶ average ▶ 2-layer MLP ▶ **768 numbers**

Reads the tags of one or more entities, without names, such as ‘shop: bakery’. The average runs over the words, and over the entities when there are several. BERT starts from its pre-training on English text. The MLP starts from random weights.

one entity → its T
a POI, or a building’s tags

several entities → one T
a street’s context, or an area

Stage 2 trains V so that a building’s V is similar to the T of its own tags. A footprint can therefore be compared directly with a text, by cosine similarity.

V · footprint encoder model.cv, projection_img, projection_s

outline image ▶ ResNet-18 ▶ linear layer ▶ mean ▶ **768 numbers**
floor area ▶ 2-layer MLP ▶ mean

The outline is drawn as a 224 × 224 image. ResNet-18 starts from ImageNet weights, and the other layers start from random weights.

P · position code function PE, not in model.pkl

latitude and longitude ▶ sin and cos at 64 frequencies ▶ **256 numbers**

A fixed formula with nothing to train, so no stage changes it. Nothing trains P to agree with T or V, so it is not compared with them. It is only joined to them for a task.

Embedding means one of these vectors. A POI’s T and a street’s T come from the same weights, so they can be compared with each other as well.

2 Training · three stages in turn

Each stage is a matching quiz on unlabelled data. It trains one encoder and leaves the other unchanged, and it starts from the weights saved by the stage before.

Start BERT-base and ImageNet ResNet-18, with new MLP and linear layers

1 Infomax · an entity ↔ its street — 625 steps

T of one entity on a street ⟷ **T** of that street’s whole context

Trains T (BERT and its MLP). V is not used.

2 Polygon · a footprint ↔ its own tags — 1,250 steps

V of a building’s footprint and area ⟷ **T** of the same building’s tags

Trains V. T is used but frozen.

3 Relation · a street ↔ a similar street — 270 steps

T of street a’s context ⟷ **T** of street b’s context, with about as many bus routes

Trains T again. V is not used.

Saved as model.pkl T as stage 3 left it, V as stage 2 left it

V was fitted to T as it was after stage 1. Stage 3 then changed T, at a low learning rate (10^{-5}) for 270 steps, and V was not refitted. The footprint tests in section 9 use the final model, so any mismatch is already in their scores.

3 Embeddings · computed after training

The model is frozen. Each kind of object gets the vectors that apply to it, from the input shown in each cell. An object may therefore have one, two or three embeddings.

	T text	V footprint	P position
POI a tagged node	its tags	–	its point
Building with tags	its tags	its footprint	its centroid
Untagged building building-function task	entities near it	its footprint	its centroid
Road segment †	its context (within 100 m)	–	a vertex
Area Leeds population test	all entities inside	mean of its footprints’ V	its centroid

The maps in sections 8e and 9d instead combine the separate T of each POI, or the V of each footprint, within each hexagon.

For a task, an object’s vectors are joined end to end

T · 768 | **V** · 768 | **P** · 256 = 1,792

numbers for a building or an area (1,024 for a POI or a street, which have no V)

A small model is trained on labelled examples for that task
an MLP in the shipped code, ridge regression in the Leeds population test

a prediction, such as people per km²

CityFM itself is not changed by a task.

In short. There is one model, not three. T, V and P are three kinds of vector. T and V come from the model’s two trained encoders, and a V can be compared with a T because stage 2 trained V to match the T of a building’s own tags. P comes from a fixed formula outside the model. A POI, building, street or area is described by whichever of the three apply, and a task joins them.

† encoder.py has no text encoding for road segments, and the shipped speed task does not call the model. The notebook computes segment context vectors itself. **Not shown.** model.pkl also holds three small layers that no stage trains or uses. **Where the paper differs.** The paper trains on the sum of the three losses at once, while this code runs the stages in turn. The paper describes a context by the mean of its entities’ T (Eq. 3), while the code averages BERT’s word vectors over all the entities and applies the MLP once. The paper’s V has an MLP after ResNet-18 and another after the mean (Eqs. 4 and 6), where the code has one linear layer and nothing after the mean.